Mini-project: a falling-sand sandbox

Introduction to Programming — Inheritance

Estimated time: 4 hours ·

Overview

You will build a small *falling-sand* simulator: a rectangular world of cells in which sand piles up, water spreads, oil floats on water, wood catches fire and smoke rises and fades away. The world is printed in the terminal as ASCII art, one frame per tick, so you can watch it evolve.

The interesting part is not the physics, which is deliberately naive, but the *shape of the code*. Each kind of matter is a class in an inheritance hierarchy. The world only ever manipulates variables of type `Element`, and every difference in behaviour comes from redefining methods (dynamic binding). The project exercises, in order of appearance:

- the *is-a* relation and the definition of sub-classes with `extends` (lesson 10);
- access rights `private`, `protected`, `public` in a hierarchy (lesson 11);
- redefinition of methods, `super.`*method*, and attribute hiding, i.e. *masquage* (lesson 12);
- constructors in a hierarchy, `super(...)`, `this(...)`, the order of construction (lesson 13);
- static versus dynamic binding and heterogeneous collections (lesson 14);
- and, from earlier weeks: encapsulation, copy constructors, `toString`, reference semantics, two-dimensional arrays and strings.

Time budget.

Part	Content	Time
A	Warm-up on paper: predict, then justify	30 min
B	Elements and the world	35 min
C	Falling particles	40 min
D	Liquids	40 min
E	Fire and smoke	45 min
F	Parallel futures: copying a world	30 min
G	Your own scene, design table, reflection	20 min
	Total	4 h

How to check yourself. Every rule in Section 2 is deterministic: there is no randomness. The expected renders shown in each part (the *checkpoints*) were produced by a reference implementation of exactly this specification. If your output differs, find the *first* tick at which it differs: that tick and that cell are your best debugging clue. Questions are numbered Q1, Q2, ...; answer them in a file `answers.txt` *before* looking anything up. A star* marks a question that goes slightly beyond what the slides state explicitly and asks you to reason from the rules you know.

Design constraints. These are part of the specification and are what the project actually tests. Re-read them at the start of every part.

R1. Course material only. Use only BOOC I and weeks 1–3 of BOOC II. In particular: no `abstract`, no `interface`, no `instanceof`, no casts between class types, no `getClass()`, no `static` attributes, no exceptions (except in the helper of Appendix B). Static methods are allowed only in your main class, as in BOOC I. If you feel that one of these would make the code better, write a comment saying where and why: this is a valuable observation for weeks 4–6.

R2. Encapsulation. All attributes are `private`. If you declare any member `protected`, justify it in a one-line comment next to it.

R3. No type tests. `World`, `Particle`, `Liquid`, `Gas` and `Fire` never test what kind of element they are dealing with: no comparison of `getName()` or `getSymbol()` to decide behaviour (the only exceptions are `World.count` and the parsing method). Differences in behaviour must come from redefinition.

R4. Concrete classes are named in few places. The names `Air`, `Wall`, `Wood`, `Fire`, `Sand`, `Water`, `Oil`, `Smoke` may appear only in: the parsing method of `World`; `Fire` (which creates `Fire` and `Smoke`); `Gas` (which creates `Air`); each class's own constructors and `copy()` method; and your main class.

R5. No duplication. The falling algorithm of Section 2.4 is written exactly once. Each `toString` that extends another one uses `super.toString()`.

R6. Constructors. `Element` has no default constructor. Every constructor of every sub-class begins with an explicit `super(...)` or `this(...)`.

1 Part A — Warm-up on paper (30 min)

Answer without a computer, then check by compiling. For every answer, write *one sentence of justification* naming the rule you used ("static type", "dynamic type", "implicit `super()`", ...). A correct output with a wrong reason counts as wrong.

A1. Order of construction

```
class A {
    public A()      { System.out.print("A"); }
    public A(int x) { this(); System.out.print("a" + x); }
}
class B extends A {
    public B()      { System.out.print("B"); }
    public B(int x) { super(x + 1); System.out.print("b" + x); }
}
class C extends B {
    public C()      { super(2); System.out.print("C"); }
}
```

Q1. What do `new C()` and `new B()` print?

Q2. The constructor `A()` is deleted and nothing else changes. Which lines no longer compile, and why? Which constructors are unaffected?

A2. A band of instruments

```
class Instrument {
    private String name;
    public Instrument(String name) { this.name = name; }
    public String getName()  { return name; }
    public String describe() { return "instrument " + name; }
    public String play()     { return "..."; }
    public String concert()  { return describe() + " plays " + play(); }
}
class StringInstrument extends Instrument {
    private int strings;
    public StringInstrument(String name, int strings) {
        super(name);
        this.strings = strings;
    }
    public String describe() {
        return super.describe() + " with " + strings + " strings";
    }
}
class Guitar extends StringInstrument {
    public Guitar() { super("guitar", 6); }
    public String play() { return "strum"; }
}
class Harp extends StringInstrument {
    public Harp() { super("harp", 47); }
    public String describe() { return "a big " + super.describe(); }
    public String play()     { return "pling" + super.play(); }
}
class Drum extends Instrument {
    public Drum() { super("drum"); }
    public String play() { return "boom"; }
}
// in main:
Instrument[] band = { new Guitar(), new Harp(), new Drum() };
for (int i = 0; i < band.length; ++i) {
    System.out.println(band[i].concert());
}
```

Q3. Give the three printed lines. For the second line, list in order every method body that is executed, naming its class.

Q4. `concert()` is written once, in `Instrument`, a class that knows nothing about harps. Explain precisely why it nevertheless calls `Harp`'s `describe()`.

Q5. Does the statement `StringInstrument s = band[0];` compile? `band[0]` is a guitar, after all.

Q6. The constructor of `Drum` is deleted. Does the program still compile? What if, in addition, `Instrument` received a second constructor `Instrument()`?

A3. Hiding an attribute*

```
class P {
    protected int x = 1;
    public int getX() { return x; }
```

```
}
class Q extends P {
    private int x = 2;                          // masquage
    public int getX() { return x; }
    public int sum()  { return x + super.x + super.getX(); }
}
// in main (same package):
P p = new Q();
Q q = new Q();
System.out.println(p.getX());
System.out.println(p.x);
System.out.println(q.sum());
```

Q7. Predict the three lines. Which of them depend on the static type of `p`, and which on its dynamic type? Use your answer to formulate a rule: "methods are ..., attributes are ...". Then explain why the course advises against *masquage*.

A4. `protected` across packages

Four files, in two packages `zoo` and `park`.

```
// file zoo/Animal.java
package zoo;
public class Animal {
    protected int energy;
    private int age;
    public int getAge() {
        return age;
    }
}
```

```
// file park/Lion.java
package park;
import zoo.Animal;
public class Lion extends Animal {
    public void eat() {
        energy = energy + 5; // (1)
        age = age + 1;       // (2)
    }
}
```

```
// file park/Keeper.java
package park;
public class Keeper {
    public void feed(Lion l) {
        l.energy += 1;       // (3)
        int a = l.getAge();  // (4)
    }
}
```

```
// file zoo/Vet.java
package zoo;
import park.Lion;
public class Vet {
    public void check(Lion l) {
        l.energy = 0;        // (5)
    }
}
```

Q8. Which of the lines (1)–(5) compile? Justify each one with the rule from lesson 11.

A5. Modelling

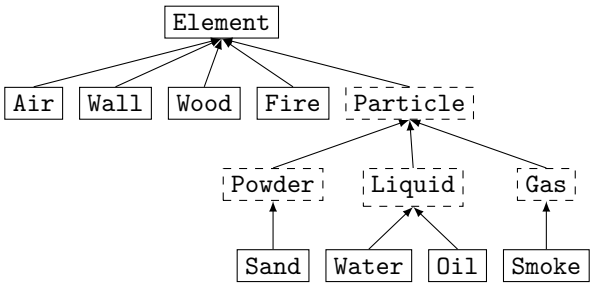
Q9. For each pair, decide whether the right relation is *is-a* (inheritance), *has-a* (an attribute) or *neither*, and justify in one line: (`World`, `Element`); (`Water`, `Liquid`); (`Sand`, `Element`); (`Fire`, `Smoke`); (`Liquid`, `Powder`); (`Wood`, `Fire`). For (`Liquid`, `Powder`), note that a liquid does fall exactly like a powder before flowing sideways: is reusing code a sufficient reason to inherit?

2 The world: specification

2.1 Grid, ticks and elements

The world is a grid of R rows and C columns. Cell (r, c) has row $r \in \{0, \ldots, R-1\}$ and column $c \in \{0, \ldots, C-1\}$; row 0 is at the *top*. Every cell always contains exactly one element; empty space is the element `Air`, never `null`. Positions outside the grid do not exist: any move or neighbour outside the grid is simply ignored.

Time advances in *ticks*. The world stores the current tick, which is 0 after construction and is incremented by one at the beginning of each step.



Arrows point to the super-class. Dashed classes never appear directly in a scene: they hold behaviour shared by their sub-classes.

Class	name	symbol	density	fluid	flammable	extinguishes	other
`Air`	`air`	␣	2	yes	no	no	
`Wall`	`wall`	`#`	100	no	no	no	
`Wood`	`wood`	`=`	30	no	yes, burn 10	no	
`Fire`	`fire`	`*`	0	no	no	no	6 ticks by default
`Sand`	`sand`	`.`	10	no	no	no	
`Water`	`water`	`~`	6	yes	no	yes	spread 3
`Oil`	`oil`	`o`	4	yes	yes, burn 3	no	spread 1
`Smoke`	`smoke`	`"`	1	yes	no	no	lifetime 8

The symbol of `Air` is a space. Any character not in this table is read as air.

2.2 Public interface of `Element`

```
public Element(String name, char symbol, int density)
public String  getName()
public char    getSymbol()
public int     getDensity()
public void    markUpdated(int tick)    // "I was updated at this tick"
public boolean wasUpdatedAt(int tick)   // true iff last mark == tick
public boolean isFluid()                // see the table
public boolean isFlammable()
public int     getBurnTime()            // 0 for non-flammable elements
public boolean extinguishesFire()
public boolean update(World w, int row, int col)
public String  toString()
```

`update` is called by the world when it is this element's turn to act; the element is at (row, col) in `w`. It returns `true` if and only if, during this call, the element moved to another cell or was

replaced. In `Element`, it does nothing and returns `false`: this is the general behaviour of matter that never acts (air, walls, wood).

`toString()` returns *name*␣`'`*symbol*`'`␣`d=`*density*, for instance `wall '#' d=100`. Some sub-classes extend this text (Parts D and E).

2.3 Public interface of `World`

```
public World(String[] lines)           // one string per row, equal lengths
public int     getRows()
public int     getCols()
public int     getTick()
public boolean isInside(int row, int col)
public Element get(int row, int col)    // precondition: isInside(row, col)
public void    set(int row, int col, Element e)
public void    swap(int r1, int c1, int r2, int c2)
public void    step()                   // one tick
public void    run(int n)               // n ticks
public String  renderRow(int row)       // the symbols of one row
public String  render()                 // all rows, each followed by '\n'
public int     count(String name)       // cells whose element has this name
```

Rule W1 (marking). `set(r, c, e)` stores `e` in the cell *and marks* `e` *as updated at the current tick.* `swap` exchanges two cells and marks nothing. The constructor fills the grid using `set`. Consequence: an element created during tick t acts for the first time at tick $t+1$.

Rule W2 (one step). `step()` does the following.

1. Increment the tick; call its new value t.
2. For $r = R-1$ down to 0 (bottom to top), and for $c = 0$ up to $C-1$ (left to right): let e be the element in cell (r, c) *at that moment.* If e was updated at tick t, skip it. Otherwise mark e as updated at t, then call `e.update(this, r, c)`.

The mark guarantees that an element acts at most once per tick, even if it moves to a cell that has not been scanned yet.

2.4 Particles

A `Particle` is an element that moves by swapping places with fluid neighbours. It has

```
public int     getGravity()   // +1 (falls); gases: -1 (rises)
public boolean canEnter(Element target)
public boolean tryMove(World w, int row, int col, int dRow, int dCol)
```

Rule P1 (who gives way). Let g be the gravity of particle p. Then p can enter *target* if and only if *target* is fluid and

$$g > 0 \;\Rightarrow\; d(\mathit{target}) < d(p), \qquad g < 0 \;\Rightarrow\; d(\mathit{target}) > d(p),$$

where d denotes density. `tryMove(w, r, c, dr, dc)` checks that $(r+dr, c+dc)$ is inside the grid and that the particle can enter the element there; if so it swaps the two cells and returns `true`, otherwise it returns `false` and changes nothing.

Rule P2 (general movement). The general `update` of a particle at (r, c) tries the following moves in order and stops at the first one that succeeds:

$$(g,\,0), \qquad (g,\,s), \qquad (g,\,-s), \qquad \text{where } s = \begin{cases} -1 & \text{if the current tick is even,} \\ +1 & \text{if the current tick is odd.} \end{cases}$$

That is: straight down (or up, for a gas), then diagonally, alternating which diagonal is tried first so that piles stay symmetric. It returns `true` iff a move succeeded.

3 Part B — Elements and the world (35 min)

Task B1. Write `Element` with the interface above and private attributes. The mark can be stored as the last tick at which the element was updated (initially -1).

Task B2. Write `Air` and `Wall`. Each needs only a constructor without parameters and, for `Air`, one redefinition. Look at lesson 13's `Carre`: "and that's all".

Task B3. Write `World`: a private two-dimensional array of `Element`, the tick, and the interface above. Parsing goes through a private method `Element createElement(char symbol)` (an instance method, called from the constructor). For now, it knows only `#` and the space; you will extend it as classes appear. Implement `step()` completely now, following rule W2: it does not depend on which elements exist.

Checkpoint B. Build the scene `drop` of Appendix A. Then `render()` returns the scene unchanged (with `.` read as air for now); `count("wall")` is 24; `System.out.println(w.get(0, 0))` prints `wall '#' d=100` and `w.get(1, 1)` prints `air␣'␣'␣d=2`. Explain in one line why `println` prints these texts although you never called `toString()` yourself.

Q10. `Element` has no default constructor (R6). What would go wrong, in the context of lesson 13, if it had one that set the name to `"?"`? Think of a sub-class whose author forgets to call `super(...)`.

4 Part C — Falling particles (40 min)

Task C1. Write `Particle` according to Section 2.4, then `Powder` and `Sand`. `Sand` contains nothing but its constructor. Add `.` to the parser.

Task C2. In your main class, write a static function `boolean conserves(World w, String name, int n)` that runs `w` for `n` ticks and returns whether `count(name)` is unchanged. Sand must be conserved in every scene without fire.

Checkpoint C. Scene `drop`, ticks 0 to 4:

```
t = 0      t = 1      t = 2      t = 3      t = 4
#######    #######    #######    #######    #######
#  .  #    #     #    #     #    #     #    #     #
#  .  #    #  .  #    #     #    #     #    #     #
#  .  #    #  .  #    #  .  #    #     #    #     #
#     #    #  .  #    #  .  #    #  .  #    #     #
#     #    #     #    #  .  #    #  .. #    # ... #
#######    #######    #######    #######    #######
```

Scene `hourglass` at ticks 5, 20 and 60, with `count("sand")` = 24 throughout:

$t = 5$	$t = 20$	$t = 60$
`###########`	`###########`	`###########`
`#    .....#`	`#        .#`	`#         #`
`## ......##`	`##      .##`	`##       ##`
`###.....###`	`###    .###`	`###     ###`
`####...####`	`####  .####`	`####   ####`
`#####.#####`	`#####.#####`	`##### #####`
`#### . ####`	`####.. ####`	`####...####`
`###  .  ###`	`### ....###`	`###.....###`
`##   .   ##`	`##..... ##`	`##.......##`
`#    .    #`	`# ....... #`	`#.........#`
`###########`	`###########`	`###########`

Q11. (*What changes if…?*) Suppose rule W2 scanned rows from top to bottom instead. Predict the scene `drop` after one tick, *then* check by temporarily changing your code. Why is bottom-to-top the natural order for falling matter, and which kind of particle will prefer the opposite order?

Q12. `Powder` currently adds nothing to `Particle`. Give two reasons, one about modelling and one about the future of the code, to keep it in the hierarchy.

5 Part D — Liquids (40 min)

A `Liquid` is a fluid particle with an integer *spread* (given to its constructor) and a private *direction* $\delta \in \{-1, +1\}$, initially $+1$ (to the right).

Rule L1 (liquid movement). The `update` of a liquid at (r, c):

1. tries the general movement of rule P2; if it succeeds, returns `true`;
2. otherwise *flows*: repeatedly moves one cell in direction δ, i.e. performs `tryMove(w, r,` c'`, 0,` δ`)` from its *current* column c', until a move fails or it has moved *spread* cells; if it moved at least once, returns `true`;
3. otherwise (it has not moved) reverses its direction ($\delta \leftarrow -\delta$) and flows once more as in step 2; returns whether it moved.

Task D1. Write `Liquid`, `Water` and `Oil`, and add `~` and `o` to the parser. Step 1 of rule L1 must be a call to `super.update(...)` (R5): this is the specialisation pattern of lesson 12.

Task D2. Redefine `toString()` in `Liquid` so that it appends ␣`spread=`*spread*␣`dir=`*direction* to the general text, e.g. `water '~' d=6 spread=3 dir=1`.

Checkpoint D. Scene `layers` at ticks 1, 2, 3 (then nothing changes):

$t = 1$	$t = 2$	$t = 3$
`#########`	`#########`	`#########`
`#       #`	`#       #`	`#       #`
`#.......#`	`#ooooooo#`	`#ooooooo#`
`#~~~~~~~#`	`#.......#`	`#~~~~~~~#`
`#ooooooo#`	`#~~~~~~~#`	`#.......#`
`#########`	`#########`	`#########`

Scene `dam` at ticks 1, 2, 3 and 12:

```
t = 1            t = 2            t = 3            t = 12
############     ############     ############     ############
#~  #      #     #  ~#      #     #   #      #     #   #      #
#~~~#      #     #~~ #      #     #~~~#      #     #   #      #
#~~~#      #     #~~~#      #     #~~~#      #     #~ ~#      #
#~~~~ ~    #     #~~~~~   ~ #     #~~~~   ~ ~#     #~~~~~~~~~~#
############     ############     ############     ############
```

After one tick of `dam`:

- `w.get(1, 1)` prints `water '~' d=6 spread=3 dir=-1`.

Q13. Nowhere in your code is there a line saying "oil goes up". Which rule makes oil end up above the water in `layers`? Trace what happens to one oil cell and the water cell above it at tick 2.

Q14. Is the scene `dam` ever static? Argue from rule L1, then check it with `render()`.

Q15. (*Masquage in your own code.*) A colleague adds `private int direction = -1;` to `Water`, "so that water starts by flowing to the left". Does the behaviour of water change? What does `toString()` print for a fresh `Water`? Explain using A3.

6 Part E — Fire and smoke (45 min)

Rule K1 (flammability). `Wood` and `Oil` are flammable, with burn times 10 and 3; `Water` extinguishes fire. Every other element keeps the general answers of `Element` (`false`, 0, `false`).

Rule S1 (gases). A `Gas` is a fluid particle with gravity -1, a *lifetime* (given to its constructor) and an *age*, initially 0. Its `update`: first increments its age; if the age has reached the lifetime, it replaces itself by a new `Air` (with `set`) and returns `true`; otherwise it performs the general movement of rule P2. Its `toString()` appends ␣`age=`*age*`/`*lifetime*.

Rule K2 (fire). A `Fire` has a number of remaining ticks. `Fire(int remaining)` is the main constructor; `Fire()` uses 6 and must be written with `this(...)`. The orthogonal neighbours of (r, c) are, in this order, up $(r-1, c)$, right $(r, c+1)$, down $(r+1, c)$ and left $(r, c-1)$, ignoring those outside the grid. Its `update`:

1. if some neighbour extinguishes fire, the fire replaces itself by a new `Smoke` and returns `true`;
2. otherwise, each flammable neighbour n, in the order above, is replaced by a new fire whose remaining ticks equal the burn time of n;
3. then the remaining ticks decrease by one; if they reach 0, the fire replaces itself by a new `Smoke` and returns `true`. Otherwise it returns `false`.

Its `toString()` appends ␣`left=`*remaining*.

Task E1. Write `Wood`, then add the redefinitions needed by rule K1. Remember R3 and R4: `Fire` must work for any flammable element added later without being modified.

Task E2. Write `Gas`, `Smoke` and `Fire`, and extend the parser with `=`, `*` and `"`.

Checkpoint E. Scene `cabin` at ticks 1, 2, 6:

$t = 1$

```
###############
#             #
#             #
#   =======   #
#   =     =   #
#   =  *  =   #
#   ===*===   #
###############
```

$t = 2$

```
###############
#             #
#             #
#   =======   #
#   =     =   #
#   =  *  =   #
#   ==***==   #
###############
```

$t = 6$

```
###############
#             #
#             #
#   =======   #
#   *     *   #
#   *  "  *   #
#   *******   #
###############
```

then at ticks 10, 15, 20; everything has disappeared by tick 30:

$t = 10$

```
###############
#             #
#             #
#   *******   #
#   *  "  *   #
#   *     *   #
#   *******   #
###############
```

$t = 15$

```
###############
#             #
#             #
#   *******   #
#   *"" ""*   #
#  ""    """  #
#             #
###############
```

$t = 20$

```
###############
#  """   """"#
#  "" " "  "  #
#    " "      #
#             #
#             #
#             #
###############
```

Scene `oilfire` at ticks 1, 2, 3, 6:

$t = 1$

```
###########
#         #
#*        #
#*ooooooo#
#~~~~~~~~~#
###########
```

$t = 2$

```
###########
#         #
#*        #
#*ooooooo"#
#~~~~~~~~~#
###########
```

$t = 3$

```
###########
#         #
#*       "#
#*oooooo" #
#~~~~~~~~~#
###########
```

$t = 6$

```
###########
#      """#
#"    "   #
#*ooo"    #
#~~~~~~~~~#
###########
```

After one tick of `oilfire`:

- `w.get(2, 1)` prints `fire '*' d=0 left=5`;
- `w.get(3, 1)` prints `fire '*' d=0 left=3`;
- `w.get(3, 5)` prints `oil 'o' d=4 spread=1 dir=-1`.

Explain why that oil cell has already turned around although it has not moved.

Q16. `Particle.update` calls `getGravity()`, and was written before `Gas` existed. When a smoke cell is updated, list the method bodies that execute (with their classes), and explain why `Particle` does not need to be modified for smoke to rise.

Q17. Suppose `set` did *not* mark the new element (rule W1). Give a scene of one row and a scene of one column in which a whole line of wood would burn in a single tick. Why does a fire placed at the *right* end of a row of wood not show the problem?

Q18. Fire is hot and rises: would `Fire extends Gas` be a good idea? List what `Fire` would inherit, what its `update` would have to look like, and whether the is-a test passes.

Q19. `Wood` and `Oil` both redefine the flammability methods, yet their only common super-class is `Element`. Is this duplication? Can it be removed with inheritance alone, without making `Element` know about burning? (Keep your answer: interfaces come in week 5.)

7 Part F — Parallel futures: copying a world (30 min)

Goal: from a world `w`, build an independent copy `future`, run the copy ahead, and compare it with the present. A correct copy satisfies the following property, which is your test:

for every scene and all k, n : run k ticks, copy, run both n ticks $\implies$ identical renders.

Task F1. (*Naive version.*) Write `World(World other)` so that it creates a *new* array but fills it with *the same element references* as `other`. Run the experiment: scene `cabin`, $k = 2$, $n = 5$. Then run it again with `hourglass`, $k = 2$, $n = 9$.

Q20. One experiment passes and the other fails. Explain exactly which attributes are shared between the two worlds, which method calls modify them, and why sand "gets away with it". What does this tell you about the limits of testing?

Task F2. (*Deep version.*) Give every class a copy constructor `C(C other)` that starts with `super(other)` and copies its own attributes (all state, including the mark). Every class that can appear in a scene redefines

```
public Element copy()   // new object of the same class, via the copy constructor
```

and `Element.copy()` returns `new Element(this)`. Rewrite `World(World other)` with `copy()`. All experiments must now pass.

Q21. In the copy constructor `Water(Water other)`, which calls `super(other)`, the argument has type `Water` but the constructor of `Liquid` expects a `Liquid`. Why is this legal? Could the call `new Element(other.get(r, c))` replace `copy()` in `World(World other)`? What would the copied world look like after a few ticks?

Q22. (*What changes if...?*) The copy constructor of `World` forgets to copy the tick. In `hourglass` with $n = 9$, for which values of k does the experiment detect it, and what does the difference look like? Predict first, then check with $k = 2$ and $k = 3$.

Q23. Suppose `Water` forgets to redefine `copy()`. What is the dynamic type of a copied water cell? (It depends on which of `Liquid`, `Particle` redefine `copy()`: answer for your code and for the other possibilities.) With your code, which scenes of Appendix A can detect the bug with the experiment, and which cannot? Under which condition would the same mistake in `Sand` go unnoticed in *every* possible scene?

8 Part G — Your scene, the design table, reflection (20 min)

Task G1. Using the helper of Appendix B, write a `main` that animates a scene: clear the screen, print `render()` and the tick, pause about 80 ms, step, repeat.

Task G2. Design your own scene, at least 30×15, using at least six kinds of element, that tells a small story in at most 200 ticks (for instance, an oil tank leaks, catches fire, and a water tower bursts in time, or not). Keep it in your main class as a `String[]`.

Task G3. Fill in a table with one row per class and one column per method of the `Element` interface (plus `getGravity`, `canEnter`, `tryMove`, `copy`). In each cell write D (defined here), R (redefined here) or nothing (inherited). Then answer:

Q24. Count the R entries for `update`. For each, say whether the redefinition *replaces* or *extends* (with `super`) the general behaviour.

Q25. `Element.update` has an empty body and `new Element("?", '?', 0)` is accepted by the compiler, although such an object is meaningless. Describe the two problems this causes, and the language feature you would like to fix them.

Q26. All your classes live in the same (default) package. In this setting, what does `protected` give compared with `private`, and compared with `public`? Would making the attributes of `Element` `protected` have saved you code? Would it have been worth it?

Optional extensions

Each extension is a test of your design: count how many existing classes you had to modify.

- **Steam.** When water extinguishes a fire, the fire becomes `Steam` (a gas) instead of smoke, and steam turns back into water when its lifetime ends. Rule S1 creates `Air` at the end of a gas's life: refactor `Gas` so that a sub-class can choose what it leaves behind, by redefining a single method.
- **Lava.** A liquid, as dense as sand, that ignites its flammable neighbours like a fire and turns into stone when it touches water. Which code of `Fire` would you like to reuse, and why does single inheritance prevent it?
- **Salt.** A powder that disappears when it is next to water, which becomes `SaltWater`: denser than water, so it sinks below it.

A Scenes

Copy each array into your main class. PDF viewers usually drop the spaces inside these strings when you copy them, so copy the arrays from the LaTeX source of this document instead.

```
String[] drop = {
    "#######",
    "#  .  #",
    "#  .  #",
    "#  .  #",
    "#     #",
    "#     #",
    "#######"
};
```

```
String[] hourglass = {
    "###########",
    "#.........#",
    "##.......##",
    "###.....###",
    "####...####",
    "##### #####",
    "####   ####",
    "###     ###",
    "##       ##",
    "#         #",
    "###########"
};
```

```
String[] layers = {
    "#########",
    "#.......#",
    "#~~~~~~~#",
    "#ooooooo#",
    "#       #",
    "#########"
};
```

```
String[] dam = {
    "############",
    "#~~~#      #",
    "#~~~#      #",
    "#~~~#      #",
    "#~~~       #",
    "############"
};
```

```
String[] cabin = {
    "################",
    "#              #",
    "#              #",
    "#   =======    #",
    "#   =     =    #",
    "#   =  *  =    #",
    "#   =======    #",
    "################"
};
```

```
String[] oilfire = {
    "############",
    "#          #",
    "#*         #",
    "#ooooooooo#",
    "#~~~~~~~~~#",
    "############"
};
```

B Terminal helper

Paste these two functions into your main class and treat them as a black box: `try`/`catch` belongs to week 6. If your terminal does not understand the escape sequence (some Windows consoles), print about forty empty lines instead.

```
static void clearScreen() {
    System.out.print("\033[H\033[2J");
    System.out.flush();
}

static void pause(int milliseconds) {
    try { Thread.sleep(milliseconds); } catch (InterruptedException e) { }
}
```

C Self-assessment

Concept	Lesson	Where it is tested
`extends`, is-a vs has-a, transitivity	10	Q9, Q12, Q18, the hierarchy itself
`protected`, packages	11	Q7, Q8, Q26, R2
redefinition and `super.`*method*	12	Q3, Q4, Tasks D1, D2, E1, Q24
masquage	12	Q7, Q15
`super(...)`, `this(...)`, order of construction	13	Q1, Q2, Q6, Q10, R6, rule K2, F2
static vs dynamic binding	14	Q3–Q5, Q7, Q16, Q21, Q23
copy constructors, references	week 2	F1, F2, Q20–Q23
2-D arrays, strings, loops, functions	BOOC I	B3, C2, G1, G2